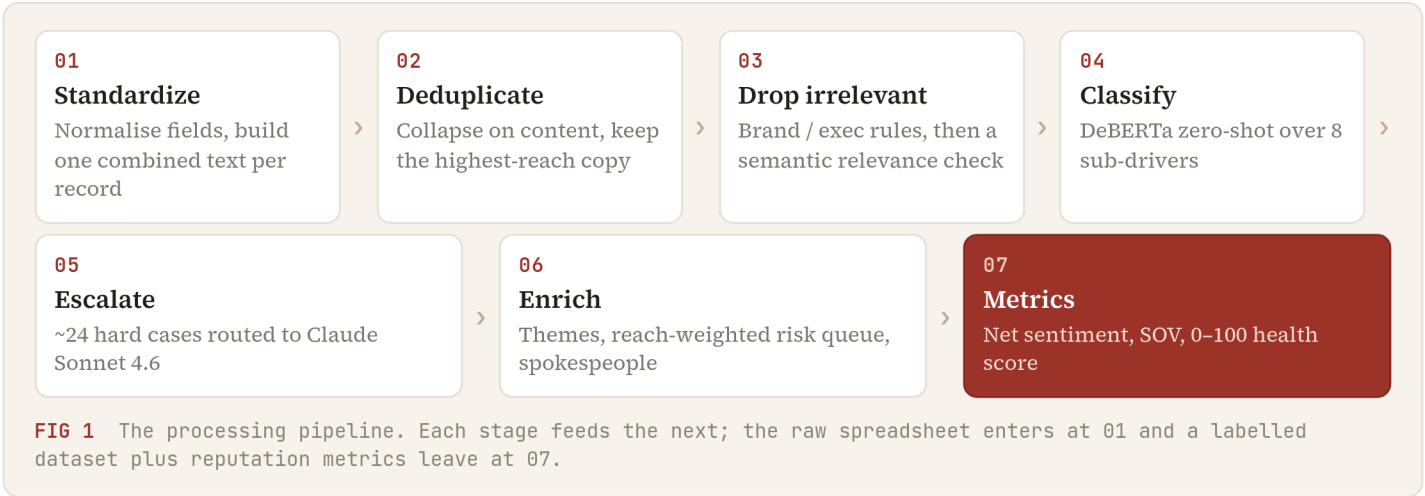


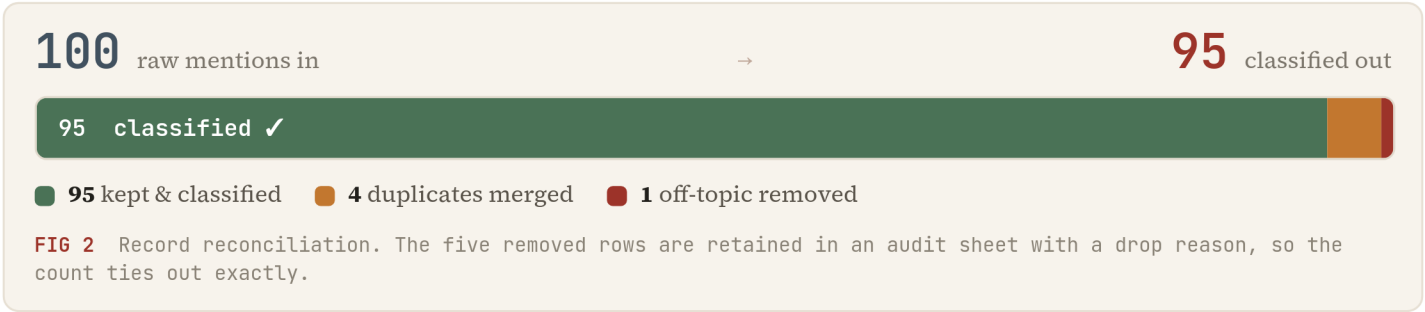
Methodology

Brand ICICI Prudential AMC / **Input** 100 digital mentions /
Output 95 cleaned, classified records + reputation metrics

The pipeline takes the raw spreadsheet and returns a labelled dataset and an executive summary, with nothing classified by hand. It runs in six stages, fully automated end to end:



The funnel is fully reconciled: 100 raw, minus 4 duplicates and 1 off-topic record, leaves 95 classified. Every removed row is kept in an audit sheet with the reason it was dropped.



Cleaning and processing

Standardizing. The provided **Sentiment** column mixes cases (**positive** , **Positive** , **Negative**), so the first step normalises it to three labels. No single text field is complete (titles are missing on 19 rows, hit-sentences on 46), so each record gets one combined text field built from the title, opening text, and hit sentence. Dates are parsed; channel, source tier, language, and a brand-salience tier are derived from the source and URL.

Deduplicating. The obvious move is to dedupe on URL, and it is wrong here: the Play Store app URL appears about fifteen times, but each row is a different review. Deduping on URL would delete real feedback. So duplicates are found on content instead, in three passes:



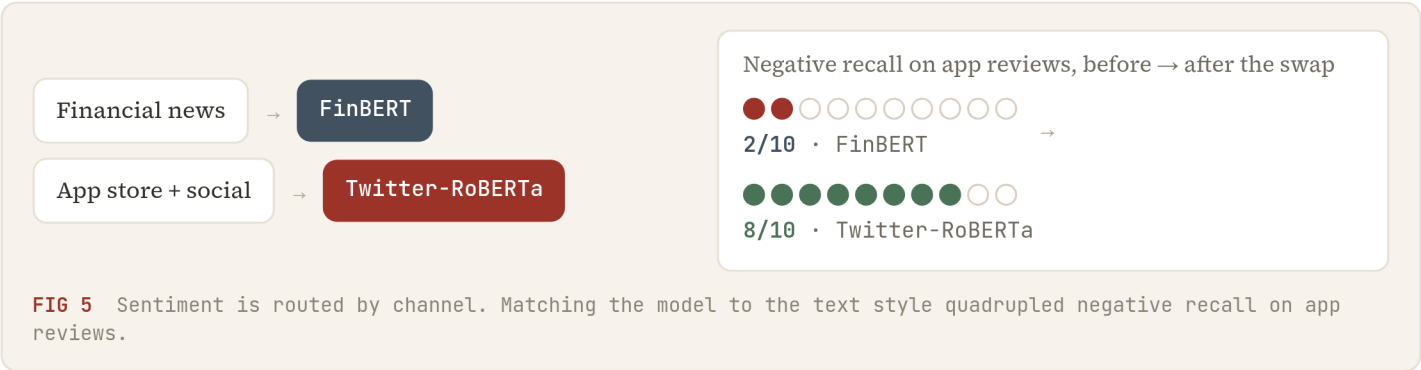
Dropping irrelevant records. A rule keeps anything that names the brand or a known executive, or comes from a first-party app/review channel. Everything else passes through a semantic relevance check against the brand. Clear noise is removed (a “Best MBA Colleges in Kalyan” article slipped into the pull); borderline “best mutual funds” listicles are kept but tagged **peripheral** so they can be excluded from headline numbers. The filter errs toward keeping, on the principle that a missed mention is worse than a low-salience one.

Classification

Driver and sub-driver use a cascade. A local zero-shot model (DeBERTa-v3) scores each record against plain-language descriptions of the eight sub-drivers and picks the best one; the parent driver is read off the sub-driver, so the two can never contradict each other. The roughly 24 records the local model is unsure about are sent to **Claude Sonnet 4.6**, which sees the full framework and returns a structured answer through forced tool use. With no API key the pipeline keeps the local labels and runs entirely offline.



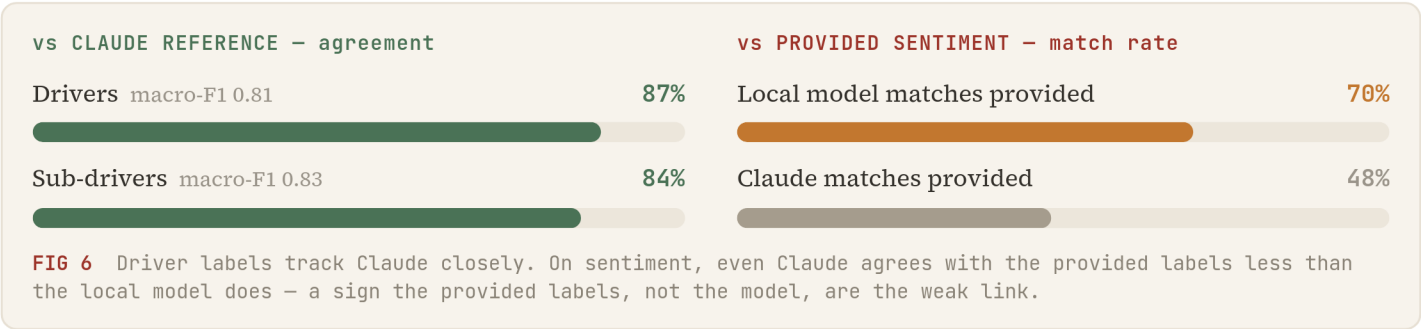
Sentiment is split by channel. FinBERT handles financial news, where it is strong, while a Twitter-tuned model handles app-store and social text, where FinBERT was reading blunt complaints as neutral. That one change lifted negative recall on app reviews from 2/10 to 8/10.



On top of the labels, the pipeline computes net sentiment, share of voice against a competitor list, a 0–100 Reputation Health Score, named discussion themes, a reach-weighted risk queue, and per-spokesperson sentiment.

How I know it works

Claude also labels all 95 records independently, which gives a reference to measure the cheap classifier against. The local model agrees with it on **87% of drivers (macro-F1 0.81)** and **84% of sub-drivers (macro-F1 0.83)**. Sentiment is a separate story: the model matches the provided labels 70% of the time, but Claude matches them only 48%, which says more about the provided labels than the model. They are a useful sanity check, not ground truth.



Tools and models

JOB	CHOICE
Data + NLP	Python, pandas, scikit-learn, HuggingFace Transformers, sentence-transformers, KeyBERT
Sentiment	FinBERT (news) + Twitter-RoBERTa (social)
Driver / sub-driver	DeBERTa-v3 zero-shot
Hard cases, validation, theme naming	Claude Sonnet 4.6, via the Anthropic SDK with forced tool use
Embeddings	MiniLM (dedup, relevance, themes)
Dashboard	Next.js, Recharts, Tailwind, on Vercel

The whole thing runs with one command and finishes in about 45 seconds on a GPU, or works on CPU.